

It is imperative to use the provided squared sheets for the simplified diagrams

An unjustified answer will be considered as false

An image is made up of a set of pixels organized as an array. For a grayscale image, each pixel is coded on a single byte. The value 0 represents the color black, the value 255 the color white. Values between 0 and 255 represent the different shades of gray. An image processing function consists in applying a certain algorithm to an array of pixels.

We would like to study and compare the execution performance of the **Threshold** function on two implementations of the Mips architecture.

The first is the classical implementation on a 5-stage pipeline presented in class. This implementation is called **Mips**.

The second, called **SS2**, is the super-scalar 2-pipeline realization studied in class.

The **Threshold** function checks whether the value of a pixel is above a certain threshold. If so, the pixel is replaced by a white pixel (code 255), and if not, by a black pixel.

```
void Threshold (unsigned char *img ,
               unsigned int   size ,
               unsigned char   threshold )
{
    for (i=0 ; i!=size ; i++)
    {
        if (img [i] >= threshold) img [i] = 255;
        else                      img [i] = 0;
    }
    return ;
}
```

Two versions of the main loop code for the **Threshold** function have been produced from Mips-32 pipeline assembler. In accordance with the conventions used by the Gcc compiler, we assume that at the loop entry :

- register R4 points to the pixel array
- register R5 contains the number of pixels in the image
- register R6 contains the threshold value

In addition, register R8 contains the address immediately after the end of the pixel array (**img + size**) and register R7 the value 255.

Version 1 :

```
_For1 :      Lbu      r10, 0 (r4 )
             Sltu    r11, r10, r6
             Sb      r0 , 0 (r4 )
             Bne     r11, r0 , _Endif
             Nop
             Sb      r7 , 0 (r4 )
_Endif :     Addiu   r4 , r4 , 1
             Bne     r4 , r8 , _For1
             Nop
```

Version 2 :

```
_For2 :      Lbu      r10, 0 (r4 )
             Sltu     r11, r10, r6
             Addiu    r11, r11, -1
             Sb       r11, 0 (r4 )
             Addiu    r4 , r4 , 1
             Bne      r4 , r8 , _For2
             Nop
```

Throughout the exercise, we assume that on average 50% of pixels are below the threshold, and that this distribution is uniform at every point in the image. The image contains 4K pixels.

First, the memory system is assumed to be perfect (all memory accesses take place in 1 cycle).

- 1- For Version 1, calculate the average number of cycles per iteration on the **Mips** implementation. You are not required to draw a diagram, but simply to indicate the stall cycles on the code. Calculate the CPI and CPI-useful.
- 2- Same question for Version 2.
- 3- For Version 1, analyze the execution of the loop using a simplified diagram for **SS2**. Assume that the **_For1** is placed on an aligned address (multiple of 8) and that the instruction buffer is empty at the loop entry. Draw a separate diagram for the case of the branch succeeds and for the case it fails. Calculate the average number of cycles per iteration. Calculate the CPI and CPI-useful.
- 4- For Version 2, analyze the execution of using a simplified scheme for **SS2**. Assume that the **_For2** is placed on an aligned address (multiple of 8) and that the instruction buffer is empty at the start of the loop. Calculate the average number of cycles per iteration. Calculate the CPI and CPI-useful.
- 5- For Version 1, optimize the loop code for **SS2** using the Software Pipeline technique.
- 6- Same question for Version 2.

For the rest of the exercise, we consider the original code of the 2 versions for the **Mips** realization.

It is now assumed that the processor is connected to a separate data and instruction cache. The instruction cache is perfect (miss rate = 0).

The data cache has a capacity of 2 Kbytes. It is a Direct Mapping cache, and a cache block contains 16 bytes.

The cache is connected to primary memory via a bus. Each cycle, 4 bytes (1 word) can be transferred on the bus. On average, 5 cycles are needed to initialize the transfer. To load a missing block into the cache, 9 cycles are required ($5 + 1 \times 16 / 4$).

- 7- How many places does the data cache contain ? How many sets ? The pixel array is made up of how many blocks ?

We wish to evaluate the execution performance of the 2 versions of the **Threshold** function under 2 hypotheses.

Firstly, we consider a "Write Through" data cache which has a 1-place posted write buffer. The buffer is emptied at the end of the bus transaction.

It is assumed that the data cache is empty at the start of the execution.

- 8- For Version 1, how many Misses occur during the processing? Calculate the number of additional cycles per iteration due to loads.

Calculate the number of additional cycles per iteration due to stores.

Deduce the average number of cycles per iteration, the CPI and the CPI-useful.

- 9- Same question for Version 2.

Now, we use "Write Back" data cache. For a store, when the block is not present in the cache, the cache first loads the missing block, then it modifies it.

- 10- For Version 1, how many Misses occur during the processing due to loads (distinguish 2 types of Miss) ? Calculate the number of additional cycles per iteration due to the Misses produced by loads.

How many Misses occur during the processing due to stores (distinguish 2 types of Miss) ?

Calculate the number of additional cycles per iteration due to stores.

Deduce the average number of cycles per iteration, the CPI and the CPI-useful.

- 11- Same question for Version 2.